

4 APRIL 2025

Structure-Level Actions in LLM- Based Coding Agents

Russell Rozenbaum

Deep Learning and Program Synthesis

LLMs have brought a resurgence in progress for automated programming techniques.

Better Context Makes Better Code Language Models Pei et al. (2023).

- They study how **cross-file** and **cross-project** information impacts argument prediction accuracy.

GraphCodeBERT Guo et al. (2020).

- Use data flow in the pre-training stage to create a pre-trained programming model that considers the inherent **structure** of code.

Structured Code Representations Enable Data-Efficient Adaptation of Code LMs

Argawal et al. (2023).

- Focus on concrete syntax trees (CSTs), representing programs as parse trees to improve pre-trained models

EXISTING WORK

Few-Shot Prompting

A method where providing few-shot examples can lead to increased performance on tasks that the LLM's training data may not effectively cover.

CoT

Chain of Thought (CoT) reasoning is widely used across LLM-based coding agents, and has been shown to greatly increase the accuracy of generated code.

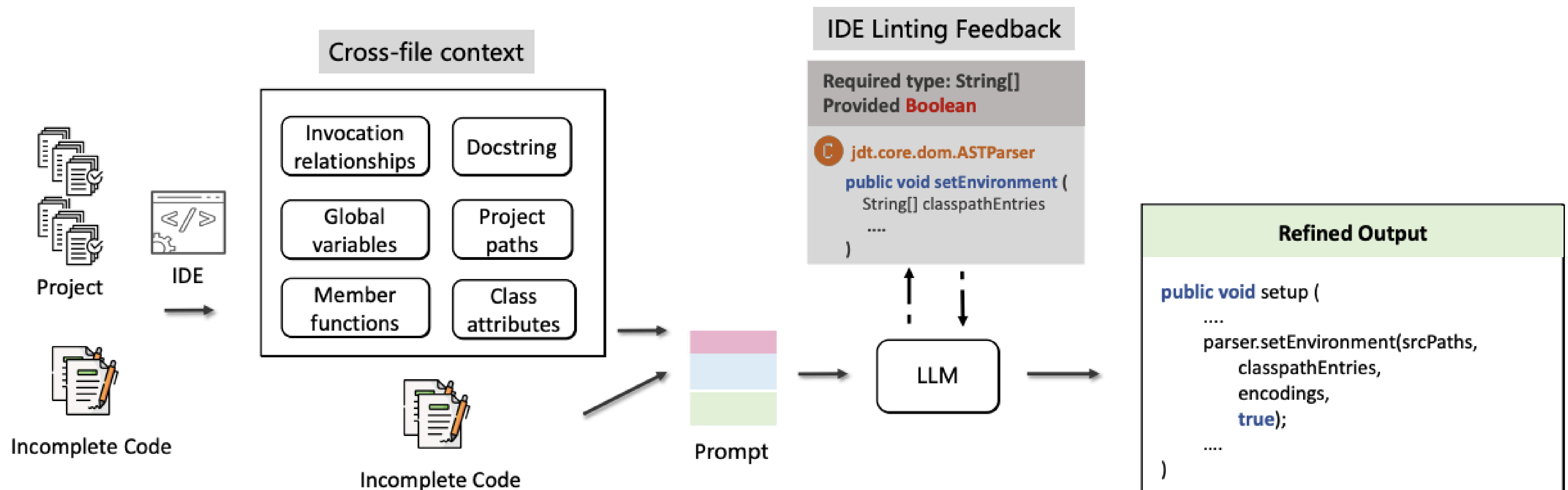
RAG

Retrieval-augmented-generation (RAG) enhances generative AI models through allowing them to access external resources and information.

Prompt Engineering Techniques

IDECoder

Li, Y. et al. offer an LLM-based code completion framework that similarly identifies and addresses the semantic contextualization problem. They integrate cross-file information, static contexts, and diagnosis results to effectively improve context-aware code completion.



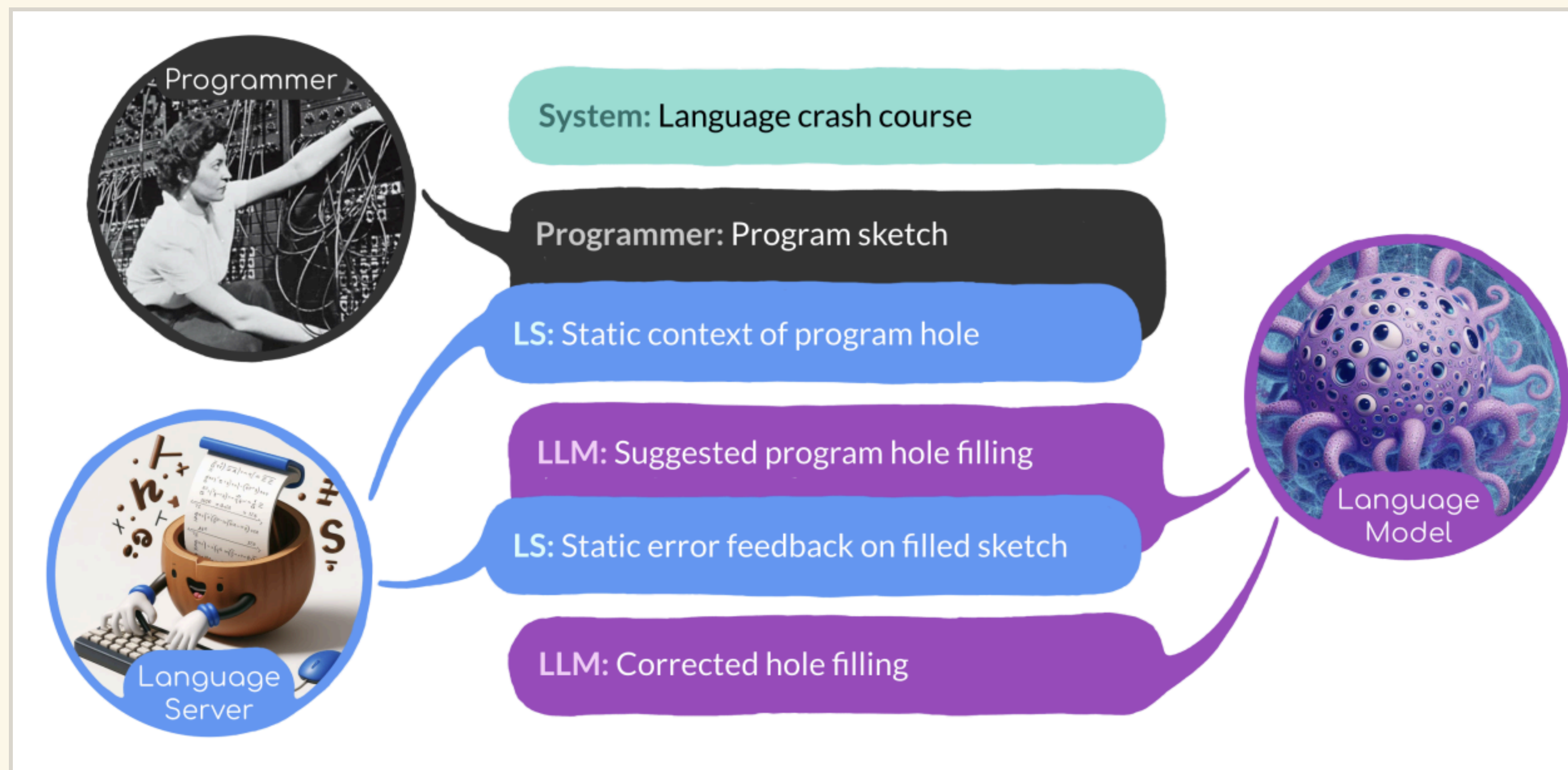
The overall framework of IDECoder.

ChatLSP

Blinn, A. et al. integrate the Hazel language server with LLM-assisted hole completions. This adequately gathers relevant type definitions and function headers of the hole the programmer has requested to be filled by the agent. Similarly to IDECoder, this allows the system to prompt the LLM with codebase-wide contextual information that is neither lexically local to the cursor nor necessarily in the same file.

Through few-shot prompting and static contextualization, they were able to bring their code-completion performance on Hazel on-par with that of TypeScript.

Their work offers a strong foundation to build our work off of.

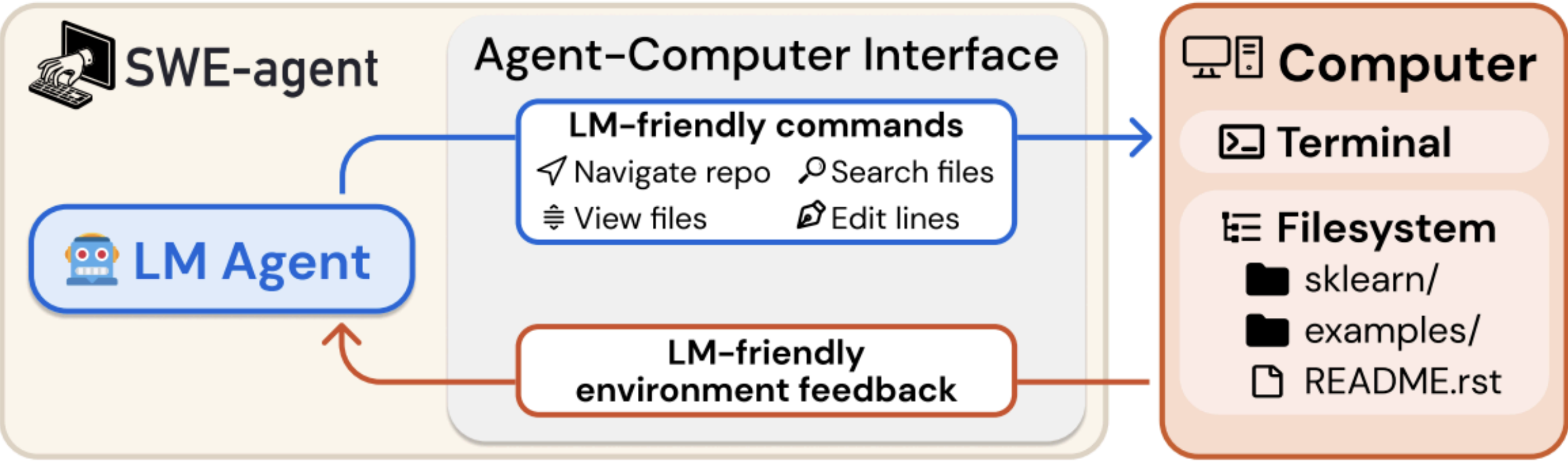


EXISTING WORK

SWE-agent

Yang & Jimenez et al. (2024) introduced SWE-agent. They equipped their LLM-based coding agent with an IDE (or the Agent-Computer-Interface (ACI)).

Their action language (depicted right) depends strongly lexical locality to be derived from line indices.



Category	Command	Documentation
<i>File viewer</i>	open <path> [<line_number>]	Opens the file at the given path in the editor. If <code>line_number</code> is provided, the window will move to include that line.
	goto <line_number>	Moves the window to show <code>line_number</code> .
	scroll_down	Moves the window up 100 lines.
	scroll_up	Moves the window down 100 lines.
<i>Search tools</i>	search_file <search_term> [<file>]	Searches for <code>search_term</code> in file. If file is not provided, searches in the current open file.
	search_dir <search_term> [<dir>]	Searches for <code>search_term</code> in all files in <code>dir</code> . If <code>dir</code> is not provided, searches in the current directory.
	find_file <file_name> [<dir>]	Finds all files with the given name in <code>dir</code> . If <code>dir</code> is not provided, searches in the current directory.
<i>File editing</i>	edit <n>: <m> <replacement_text> end_of_edit	Replaces lines <code>n</code> through <code>m</code> (inclusive) with the given text in the open file. All of the <code>replacement_text</code> will be entered, so make sure your indentation is formatted properly. Python files will be checked for syntax errors after the edit. If an error is found, the edit will not be executed. Reading the error message and modifying your command is recommended as issuing the same command will return the same error.
	create <filename>	Creates and opens a new file with the given name.
<i>Task</i>	submit	Generates and submits the patch from all previous edits and closes the shell.

Human and LLMs are Different

When designing IDEs for LLMs (aka ACIs). As Yang & Jimenez et al. point out the following in their work on SWE-agent:

- LLMs currently **lack the visual processing capabilities** that make GUI components so helpful in human-oriented IDEs.
- Humans are effective at ignoring irrelevant text and information, whereas unnecessary input-information to LLMs leads to increased **computation costs**, a **waste of memory**, and **distracting contexts**.

Or Are They?

No, they still are quite different.

However, LLMs are trained to pick up on patterns in human-generated code. Some notable patterns and behaviors in human programming are:

- **Large, high-level edit actions** effectively mitigate bugs (Afzal & Goues. 2018.)
- Use of **abstraction techniques** (chunking code into blocks, definitions, modules, etc.)
- Use of **names as opposed to line locations** when referring to the contents of a program

METHODS

1. Perception

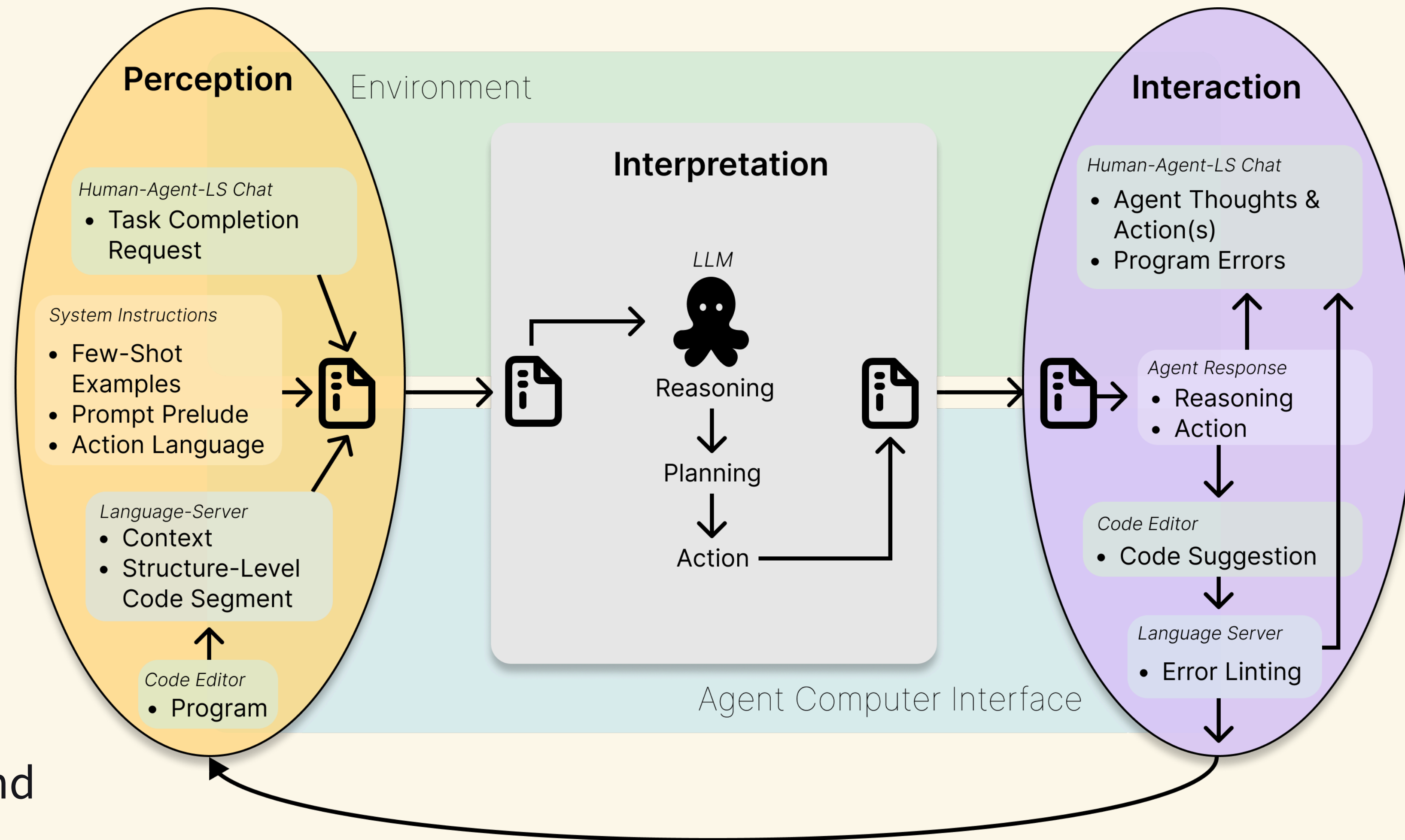
- Where the agent, aided by other tools such as a language server, gathers information about its environment

2. Interpretation

- Where the LLM stores, processes, and reasons about relevant information

3. Interaction

- Where the agent interacts with and makes changes upon the environment in which it operates



A High, Structured-Level Action Language

Category	Command	Documentation
<i>File Viewer</i>	goto_definition <variable_name>	Places the cursor at the nearest definition of the matched variable_name .
	goto_type_definition <variable_name>	Places the cursor at the nearest type definition for the matched variable_name .
	show_references <variable_name>	Displays a list of definitions where the variable matching variable_name is referenced.
	scroll_down	Moves cursor to preceding definition, if one exists.
	scroll_up	Moves cursor to succeeding definition, if one exists.
<i>File Editing</i>	replace <replacement_text>	Replaces the text of the current definition with replacement_text
<i>Task</i>	submit	Ends the iterative task completion process.

Category	Command	Documentation
<i>File viewer</i>	open <path> [<line_number>]	Opens the file at the given path in the editor. If <code>line_number</code> is provided, the window will move to include that line.
	goto <line_number>	Moves the window to show line_number.
	scroll_down	Moves the window up 100 lines.
	scroll_up	Moves the window down 100 lines.
<i>Search tools</i>	search_file <search_term> [<file>]	Searches for <code>search_term</code> in file. If file is not provided, searches in the current open file.
	search_dir <search_term> [<dir>]	Searches for <code>search_term</code> in all files in dir. If dir is not provided, searches in the current directory.
	find_file <file_name> [<dir>]	Finds all files with the given name in dir. If dir is not provided, searches in the current directory.
<i>File editing</i>	edit <n>:<m> <replacement_text> end_of_edit	Replaces lines n through m (inclusive) with the given text in the open file. All of the <code>replacement_text</code> will be entered, so make sure your indentation is formatted properly. Python files will be checked for syntax errors after the edit. If an error is found, the edit will not be executed. Reading the error message and modifying your command is recommended as issuing the same command will return the same error.
	create <filename>	Creates and opens a new file with the given name.
<i>Task</i>	submit	Generates and submits the patch from all previous edits and closes the shell.

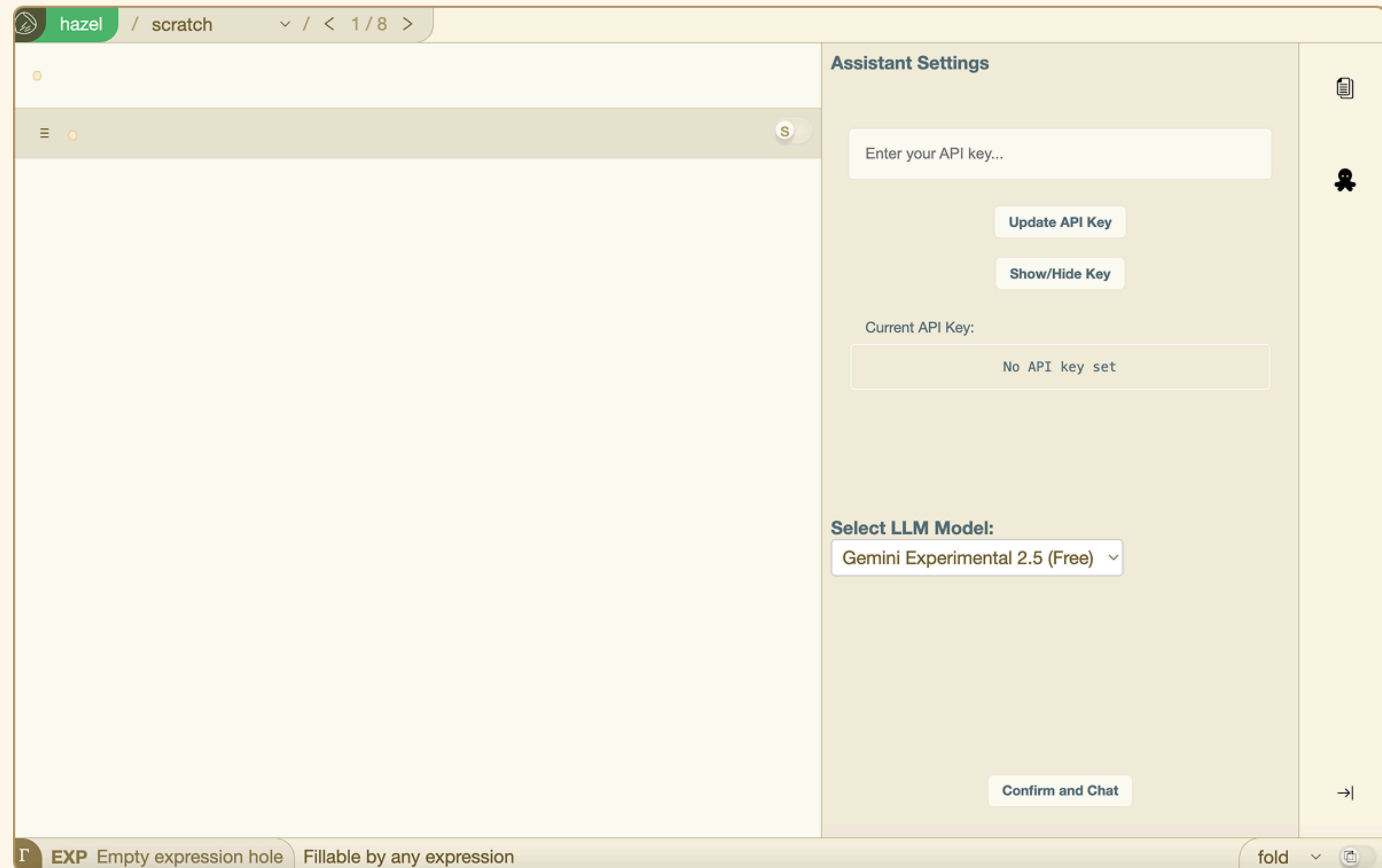
Category	Command	Documentation
<i>File Viewer</i>	goto_definition <variable_name>	Places the cursor at the nearest definition of the matched <code>variable_name</code> .
	goto_type_definition <variable_name>	Places the cursor at the nearest type definition for the matched <code>variable_name</code> .
	show_references <variable_name>	Displays a list of definitions where the variable matching <code>variable_name</code> is referenced.
	scroll_down	Moves cursor to preceding definition, if one exists.
	scroll_up	Moves cursor to succeeding definition, if one exists.
<i>File Editing</i>	replace <replacement_text>	Replaces the text of the current definition with <code>replacement_text</code>
<i>Task</i>	submit	Ends the iterative task completion process.

Agent Sidebar

We add the agent sidebar to the right-hand side of Hazel's front-end page.

Displayed right is the settings menu of the agent sidebar, where the user can:

- Enter their OpenRouter **API Key**
- Select a **large language model** available through OpenRouter and that we currently support



Three Assistant Modes

Tutor

The assistant's tutor mode isn't necessarily acting as an agent here.

Rather, we concatenate relevant Hazel documentation to give to the LLM so it can effectively answer questions about Hazel users may have.

Suggest

We integrate Blinn et al.'s work on ChatLSP and static contextualization here.

The user can type '??' or '?a' in a hole to trigger a prompt with the program sketch, relevant context, few-shot examples, and basic instructions to be sent to an LLM. If the LLM returns an error-free response, the code is suggested as opaque text at the relevant hole.

Compose

This is where we plan to implement the task completion agent, equipped with our proposed action language and ACI.

Currently, this mode is non-functional.

METHODS

Tutor

The assistant’s tutor mode isn’t necessarily acting as an agent here.

Rather, we concatenate relevant Hazel documentation to give to the LLM so it can effectively answer questions about Hazel users may have.

Tutor Mode

TutorSuggestCompose

User

What is a function definition?

Assistant

In Hazel, functions are defined using the 'let' and 'fun' keywords. A function can take a single argument (which can be a tuple if you need multiple parameters) and returns a value. Functions can also be annotated with types. Here's a basic example showing different ways to define functions:

```
let identity = fun x -> x in

let add = fun (x, y) -> x + y in

let typed_function: Int -> Bool = fun x -> x > 0 in
```

Ask a question about Hazel or anything...↑

TutorSuggestCompose

User

What are ADTs?

Assistant

ADTs (Algebraic Data Types) in Hazel are user-defined data types that allow you to create custom data structures. They are defined using the `type` keyword and can consist of one or more constructors marked with `+`. Here's a simple explanation: 1. ADTs are created using the `type` keyword followed by a name 2. Constructors are marked with `+` and can hold data 3. You can pattern match on ADTs using `case` expressions 4. They can be used to model complex data structures Here's an example of defining and using a simple ADT that represents a binary tree:

```
type Tree =
  + Leaf(Int)
  + Node(Tree, Tree)
in
let tree = Node(Leaf(1), Leaf(2)) in
case tree
| Leaf(n) => n
| Node(l, r) => 0
end
```

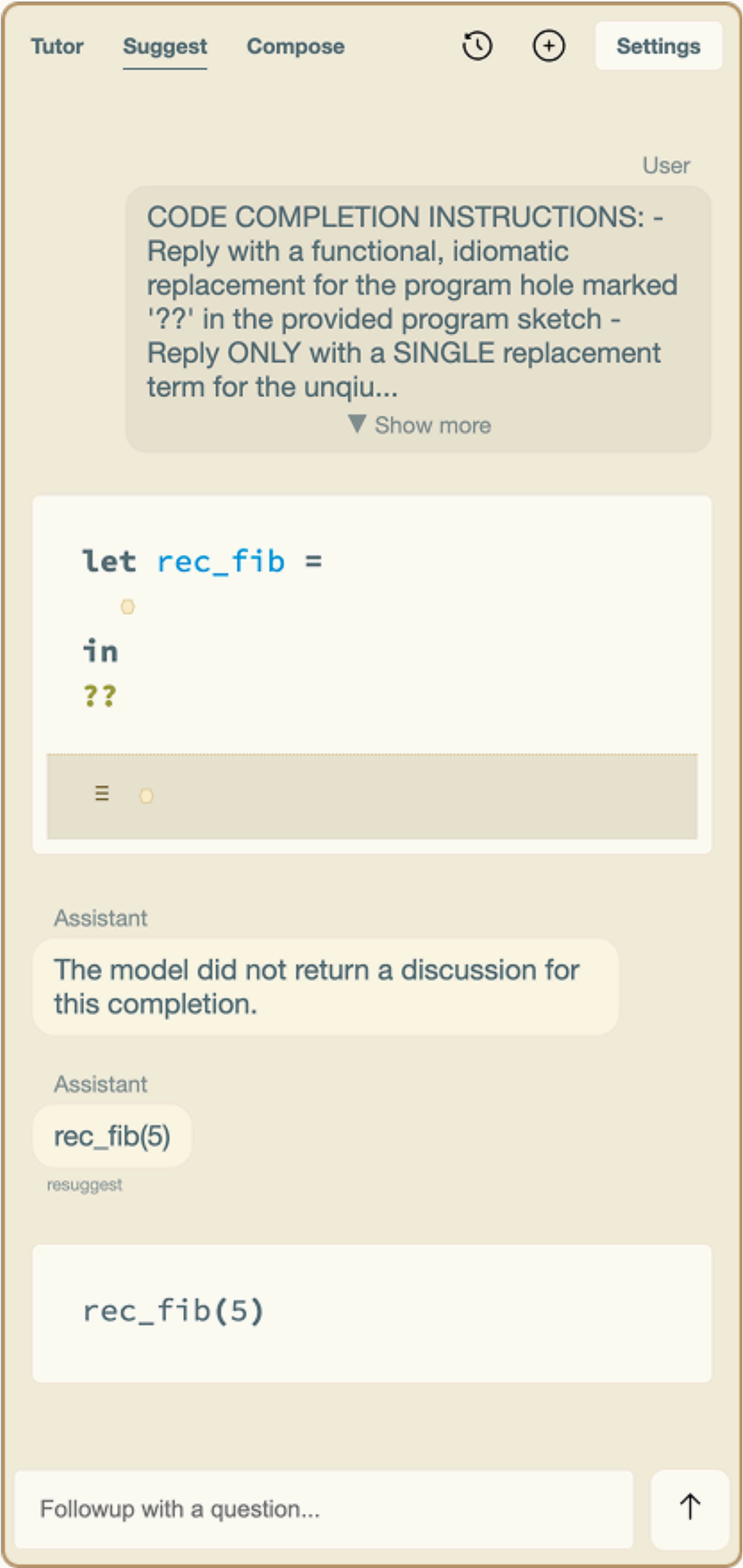
Ask a question about Hazel or anything...↑

METHODS

```
let rec_fib =  
    
in  
  ??
```

```
let rec_fib =  
    
in  
  rec_fib(5)
```

```
let rec_fib =  
    
in  
  rec_fib(5)
```



Suggest

We integrate Blinn et al.’s work on ChatLSP and static contextualization here.

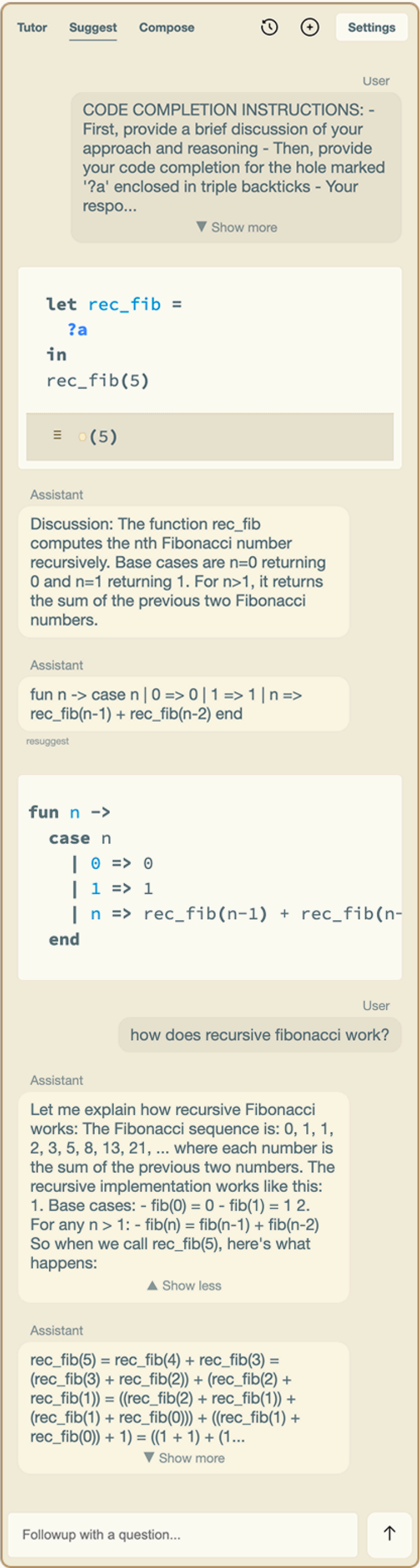
The user can type ‘??’ or ‘?a’ in a hole to trigger a prompt with the program sketch, relevant context, few-shot examples, and basic instructions to be sent to an LLM. If the LLM returns an error-free response, the code is suggested as opaque text at the relevant hole.

METHODS

```
let rec_fib =  
  ?a  
in  
  rec_fib(5)
```

```
let rec_fib =  
  fun n ->  
    case n  
    | 0 => 0  
    | 1 => 1  
    | n => rec_fib(n-1) + rec_fib(n-2)  
end  
in  
  rec_fib(5)
```

```
let rec_fib =  
  fun n ->  
    case n  
    | 0 => 0  
    | 1 => 1  
    | n => rec_fib(n-1) + rec_fib(n-2)  
end  
in  
  rec_fib(5)
```



Suggest

We integrate Blinn et al.’s work on ChatLSP and static contextualization here.

The user can type ‘??’ or ‘?a’ in a hole to trigger a prompt with the program sketch, relevant context, few-shot examples, and basic instructions to be sent to an LLM. If the LLM returns an error-free response, the code is suggested as opaque text at the relevant hole.

METHODS

```
# Function matches string input to output a sensible emoji #
let emoji_farm : String -> String =
  fun inp ->
    ?a
in
```

```
# Function matches string input to output a sensible emoji #
let emoji_farm : String -> String =
  fun inp ->
    case inp
    | "cow" => "🐮"
    | "chicken" => "🐔"
    | "pig" => "🐷"
    | "sheep" => "🐏"
    | "farm" => "🏡"
    | "tractor" => "🚜"
    | _ => "🌾"
end
in
```

```
# Function matches string input to output a sensible emoji #
let emoji_farm : String -> String =
  fun inp ->
    case inp
    | "cow" => "🐮"
    | "chicken" => "🐔"
    | "pig" => "🐷"
    | "sheep" => "🐏"
    | "farm" => "🏡"
    | "tractor" => "🚜"
    | _ => "🌾"
  end
in
```

Tutor Suggest Compose ⌚ + Settings

User

CODE COMPLETION INSTRUCTIONS: - First, provide a brief discussion of your approach and reasoning - Then, provide your code completion for the hole marked '?a' enclosed in triple backticks - Your respo...
▼ Show more

Function matches string input to output a sensible emoji #
let emoji_farm : String -> String
 fun inp ->
 ?a
in
⌵

Assistant

Discussion: The function emoji_farm maps input strings to emoji representations. A case expression is appropriate for pattern matching different animal/farm-related words to their corresponding emojis. We include common farm animals and items, with a default case for unmatched inputs.
▲ Show less

Assistant

case inp | "cow" => "🐮" | "chicken" => "🐔" | "pig" => "🐷" | "sheep" => "🐏" | "farm" => "🏡" | "tractor" => "🚜" | _ => "🌾" end
resuggest

case inp
 | "cow" => "🐮"
 | "chicken" => "🐔"
 | "pig" => "🐷"
 | "sheep" => "🐏"
 | "farm" => "🏡"
 | "tractor" => "🚜"
 | _ => "🌾"
end

Followup with a question... ⬆

Suggest

We integrate Blinn et al.’s work on ChatLSP and static contextualization here.

The user can type ‘??’ or ‘?a’ in a hole to trigger a prompt with the program sketch, relevant context, few-shot examples, and basic instructions to be sent to an LLM. If the LLM returns an error-free response, the code is suggested as opaque text at the relevant hole.

Further Work

- Implementing and Evaluating the Action Language in Hazel
- Extending the MVU Benchmark

Coding Assistants in Education

- Do generative AI code assistants have a positive effect on learning functional programming?
- On learning a new programming language's syntax and semantics?

On-the-Fly Code Completion

- Hazel's unique roots in gradual CMTT make it a good candidate for